Representing Type Theory in Type Theory

Fredrik Nordvall Forsberg

Joint work Liang-Ting Chen and Tzu-Chun Tsai

MSP101 Seminar

3 March 2025

Why do type theory in type theory?

- Metaprogramming.
- Mechanised metatheory.
- Mathematical motivation.

Why do type theory in type theory?

- Metaprogramming.
- Mechanised metatheory.
- Mathematical motivation.
- **Also, for us:** interesting example of the use of advanced inductive types.

Representing simple type theory

First we inductively define $\mathsf{Ty} : \mathsf{Set}$:

$$\begin{aligned} \mathsf{N} &: \mathsf{Ty} \\ _\Rightarrow_ &: \mathsf{Ty} \to \mathsf{Ty} \to \mathsf{Ty} \end{aligned}$$

Then we define $\mathsf{Ctx} : \mathsf{Set}$ as a (backwards) list of types:

$$\begin{aligned} \cdot &: \mathsf{Ctx} \\ _,_ &: \mathsf{Ctx} \to \mathsf{Ty} \to \mathsf{Ctx} \end{aligned}$$

Terms and substitutions

Next we can inductively define well typed terms in a given context $\mathsf{Tm} : \mathsf{Ctx} \to \mathsf{Ty} \to \mathsf{Set}$:

$$
\begin{aligned}
\mathsf{var} &: (i : S \in \Gamma) \to \mathsf{Tm}\,\Gamma\,S \\
\mathsf{zero} &: \mathsf{Tm}\,\Gamma\,\mathsf{N} \\
\mathsf{succ} &: \mathsf{Tm}\,\Gamma\,\mathsf{N} \to \mathsf{Tm}\,\Gamma\,\mathsf{N} \\
\mathsf{rec} &: (S : \mathsf{Ty}) \to \mathsf{Tm}\,\Gamma\,S \to \mathsf{Tm}\,(\Gamma, \mathsf{N}, S)\,S \to \mathsf{Tm}\,\Gamma\,\mathsf{N} \to \mathsf{Tm}\,\Gamma\,S \\
\mathsf{lam} &: \mathsf{Tm}\,(\Gamma, S)\,T \to \mathsf{Tm}\,\Gamma(S \Rightarrow T) \\
\mathsf{app} &: \mathsf{Tm}\,\Gamma(S \Rightarrow T) \to \mathsf{Tm}\,(\Gamma, S)\,T
\end{aligned}
$$

Finally we can use terms to define substitutions $\mathsf{Sub} : \mathsf{Ctx} \to \mathsf{Ctx} \to \mathsf{Set}$, and a reduction relation.

(Of course, other choices are possible, e.g. a different representation of variables.)

Terms and substitutions

Next we can inductively define well typed terms in a given context $\mathsf{Tm} : \mathsf{Ctx} \to \mathsf{Ty} \to \mathsf{Set}$:

$$
\begin{aligned}
\mathsf{var} &: (i : S \in \Gamma) \to \mathsf{Tm}\,\Gamma\,S \\
\mathsf{zero} &: \mathsf{Tm}\,\Gamma\,\mathsf{N} \\
\mathsf{succ} &: \mathsf{Tm}\,\Gamma\,\mathsf{N} \to \mathsf{Tm}\,\Gamma\,\mathsf{N} \\
\mathsf{rec} &: (S : \mathsf{Ty}) \to \mathsf{Tm}\,\Gamma\,S \to \mathsf{Tm}\,(\Gamma, \mathsf{N}, S)\,S \to \mathsf{Tm}\,\Gamma\,\mathsf{N} \to \mathsf{Tm}\,\Gamma\,S \\
\mathsf{lam} &: \mathsf{Tm}\,(\Gamma, S)\,T \to \mathsf{Tm}\,\Gamma(S \Rightarrow T) \\
\mathsf{app} &: \mathsf{Tm}\,\Gamma(S \Rightarrow T) \to \mathsf{Tm}\,(\Gamma, S)\,T
\end{aligned}
$$

Finally we can use terms to define substitutions $\mathsf{Sub} : \mathsf{Ctx} \to \mathsf{Ctx} \to \mathsf{Set}$, and a reduction relation.

(Of course, other choices are possible, e.g. a different representation of variables.)

Note that since our reduction relation is just a relation, not a real equality, we have to be careful to treat "equal" terms the same.

Summary of dependencies

For simple types: $\mathsf{Ty} \rightsquigarrow \mathsf{Ctx} \rightsquigarrow \mathsf{Tm} \rightsquigarrow \mathsf{Sub} \rightsquigarrow\ =$.

Summary of dependencies

For simple types: $\mathsf{Ty} \rightsquigarrow \mathsf{Ctx} \rightsquigarrow \mathsf{Tm} \rightsquigarrow \mathsf{Sub} \rightsquigarrow\ =$.

For dependent types:

$$\begin{tikzcd}
 & \mathsf{Tm} & \\
\mathsf{Ty} & & \mathsf{Sub} \\
 & \mathsf{Ctx} \arrow[uul, squiggly] \arrow[uur, squiggly] & =
\end{tikzcd}$$

Summary of dependencies

For simple types: $\mathsf{Ty} \rightsquigarrow \mathsf{Ctx} \rightsquigarrow \mathsf{Tm} \rightsquigarrow \mathsf{Sub} \rightsquigarrow\ =$.

For dependent types:

$$
\begin{tikzcd}
 & \mathsf{Tm} & \\
\mathsf{Ty} & & \mathsf{Sub} \\
\mathsf{Ctx} \arrow[uu, squiggly] & = &
\end{tikzcd}
$$

Summary of dependencies

For simple types: $\mathsf{Ty} \rightsquigarrow \mathsf{Ctx} \rightsquigarrow \mathsf{Tm} \rightsquigarrow \mathsf{Sub} \rightsquigarrow\ =$.

For dependent types:

$$\begin{tikzcd}
 & \mathsf{Tm} \arrow[dl, squiggly] & \\
\mathsf{Ty} & & \mathsf{Sub} \\
 & \mathsf{Ctx} \arrow[ul, squiggly] & =
\end{tikzcd}$$

Summary of dependencies

For simple types: $\mathsf{Ty} \rightsquigarrow \mathsf{Ctx} \rightsquigarrow \mathsf{Tm} \rightsquigarrow \mathsf{Sub} \rightsquigarrow\ =$.

For dependent types:

$$
\begin{tikzcd}
 & \mathsf{Tm} \arrow[dl, squiggly] \\
\mathsf{Ty} & \mathsf{Sub} \arrow[l, squiggly] \\
\mathsf{Ctx} \arrow[u, squiggly] & = \arrow[ul, squiggly]
\end{tikzcd}
$$

Summary of dependencies

For simple types: $\mathsf{Ty} \rightsquigarrow \mathsf{Ctx} \rightsquigarrow \mathsf{Tm} \rightsquigarrow \mathsf{Sub} \rightsquigarrow\ =$.

For dependent types:

$$
\begin{tikzcd}
 & \mathsf{Tm} \arrow[dl, squiggly] \\
\mathsf{Ty} \arrow[dr, squiggly, leftrightarrow] & \mathsf{Sub} \arrow[l, squiggly] \\
 & \mathsf{Ctx} \arrow[ur, squiggly] & =
\end{tikzcd}
$$

Summary of dependencies

For simple types: $\mathsf{Ty} \rightsquigarrow \mathsf{Ctx} \rightsquigarrow \mathsf{Tm} \rightsquigarrow \mathsf{Sub} \rightsquigarrow\ =$.

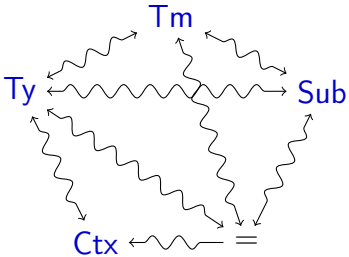
For dependent types:

$$
\begin{tikzcd}
 & \mathsf{Tm} \arrow[dl, leftrightsquigarrow] \arrow[dr, leftrightsquigarrow] \arrow[ddl, leftrightsquigarrow] \arrow[ddr, leftrightsquigarrow] & \\
\mathsf{Ty} \arrow[rr, leftrightsquigarrow] \arrow[dr, leftrightsquigarrow] \arrow[ddr, leftrightsquigarrow] & & \mathsf{Sub} \arrow[dl, leftrightsquigarrow] \arrow[ddl, leftrightsquigarrow] \\
 & = \arrow[d, leftrightsquigarrow] & \\
\mathsf{Ctx} & &
\end{tikzcd}
$$

Summary of dependencies

For simple types: $\mathsf{Ty} \rightsquigarrow \mathsf{Ctx} \rightsquigarrow \mathsf{Tm} \rightsquigarrow \mathsf{Sub} \rightsquigarrow\ =$.

For dependent types:



We need to define everything simultaneously with everything else.

Successes so far

Danielsson [2007] represented type theory in AgdaLight.

- Used some foundationally underexplored aspects of the system.
- Explicit equality relation, and proves that all constructions respect it (an instance of "setoid hell").

Successes so far

Danielsson [2007] represented type theory in AgdaLight.

- Used some foundationally underexplored aspects of the system.
- Explicit equality relation, and proves that all constructions respect it (an instance of "setoid hell").

Chapman [2009] represented type theory in Agda 2.

- Foundationally more straightforward ("only" inductive-inductive definitions).
- Still an explicit equality relation, with manual proof effort.

Successes so far

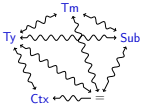
Danielsson [2007] represented type theory in AgdaLight.

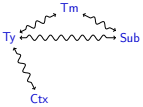
- Used some foundationally underexplored aspects of the system.
- Explicit equality relation, and proves that all constructions respect it (an instance of "setoid hell").

Chapman [2009] represented type theory in Agda 2.

- Foundationally more straightforward ("only" inductive-inductive definitions).
- Still an explicit equality relation, with manual proof effort.

Altenkirch and Kaposi [2016] used quotient-inductive-inductive types.

- Equality in object theory is "real" equality in host theory.
- All constructions automatically respect object equality.
- Cannot treat internally equal terms differently.
- Takes us from 

to 

- However, we will see that there is still a "transport hell" to tackle.

Representing the substitution calculus

We simultaneously define the following inductively:

$$
\begin{aligned}
\mathsf{Ctx} &: \mathsf{Set} \\
\mathsf{Sub} &: \mathsf{Ctx} \to \mathsf{Ctx} \to \mathsf{Set} \\
\mathsf{Ty} &: \mathsf{Ctx} \to \mathsf{Set} \\
\mathsf{Tm} &: (\Gamma : \mathsf{Ctx}) \to \mathsf{Ty}\ \Gamma \to \mathsf{Set}
\end{aligned}
$$

Representing the substitution calculus

We simultaneously define the following inductively:

$$\begin{aligned}
\mathsf{Ctx} &: \mathsf{Set} \\
\mathsf{Sub} &: \mathsf{Ctx} \to \mathsf{Ctx} \to \mathsf{Set} \\
\mathsf{Ty} &: \mathsf{Ctx} \to \mathsf{Set} \\
\mathsf{Tm} &: (\Gamma : \mathsf{Ctx}) \to \mathsf{Ty}\ \Gamma \to \mathsf{Set}
\end{aligned}$$

Contexts are defined by the following constructors:

$$\begin{aligned}
\cdot &: \mathsf{Ctx} \\
_,_ &: (\Gamma : \mathsf{Ctx}) \to \mathsf{Ty}\ \Gamma \to \mathsf{Ctx}.
\end{aligned}$$

For now, we keep types simple. We will extend them later.

$$\begin{aligned}
\mathcal{U} &: \mathsf{Ty}\,\Gamma \\
_\,[_] &: \mathsf{Ty}\ \Delta \to \mathsf{Sub}\ \Gamma\ \Delta \to \mathsf{Ty}\ \Gamma.
\end{aligned}$$

Substitutions

Substitutions $\Gamma \to \cdot, A_1, \ldots, A_n$ can be given by lists of terms $\Gamma \vdash t_i : A_i$:

$$\cdot : \mathsf{Sub}\ \Gamma\ \cdot$$

$$_,_ : (\sigma : \mathsf{Sub}\ \Gamma\ \Delta) \to \mathsf{Tm}\ \Gamma\ (A\,[\sigma]) \to \mathsf{Sub}\ \Gamma\ (\Delta, A)$$

Substitutions

Substitutions $\Gamma \to \cdot, A_1, \ldots, A_n$ can be given by lists of terms $\Gamma \vdash t_i : A_i$:

$$\begin{aligned} \cdot &: \mathsf{Sub}\ \Gamma\ \cdot \\ _,_ &: (\sigma : \mathsf{Sub}\,\Gamma\,\Delta) \to \mathsf{Tm}\ \Gamma\ (A\,[\sigma]) \to \mathsf{Sub}\ \Gamma\ (\Delta, A) \end{aligned}$$

We also want identities and composition:

$$\begin{aligned} \mathsf{id} &: \mathsf{Sub}\ \Gamma\ \Gamma \\ _;_ &: \mathsf{Sub}\ \Gamma\ \Delta \to \mathsf{Sub}\ \Delta\ \Theta \to \mathsf{Sub}\ \Gamma\ \Theta \end{aligned}$$

Substitutions

Substitutions $\Gamma \to \cdot, A_1, \ldots, A_n$ can be given by lists of terms $\Gamma \vdash t_i : A_i$:

$$\begin{aligned} \cdot &: \mathsf{Sub}\ \Gamma\ \cdot \\ _,_ &: (\sigma : \mathsf{Sub}\ \Gamma\ \Delta) \to \mathsf{Tm}\ \Gamma\ (A\,[\sigma]) \to \mathsf{Sub}\ \Gamma\ (\Delta, A) \end{aligned}$$

We also want identities and composition:

$$\begin{aligned} \mathsf{id} &: \mathsf{Sub}\ \Gamma\ \Gamma \\ _;_ &: \mathsf{Sub}\ \Gamma\ \Delta \to \mathsf{Sub}\ \Delta\ \Theta \to \mathsf{Sub}\ \Gamma\ \Theta \end{aligned}$$

As well as weakening, variables, and term substitution:

$$\pi_1 : \mathsf{Sub}\ \Gamma\ (\Delta, A) \to \mathsf{Sub}\ \Gamma\ \Delta$$

$$\begin{aligned} \pi_2 &: (\sigma : \mathsf{Sub}\ \Gamma\ (\Delta, A)) \to \mathsf{Tm}\ \Gamma\ (A\,[\pi_1\,\sigma]) \\ _\,[_] &: \mathsf{Tm}\ \Delta\ A \to (\sigma : \mathsf{Sub}\ \Gamma\ \Delta) \to \mathsf{Tm}\ \Gamma\ (A\,[\sigma]). \end{aligned}$$

Substitutions

Substitutions $\Gamma \to \cdot, A_1, \ldots, A_n$ can be given by lists of terms $\Gamma \vdash t_i : A_i$:

$$\begin{aligned} \cdot &: \mathsf{Sub}\ \Gamma\ \cdot \\ _,_ &: (\sigma : \mathsf{Sub}\ \Gamma\ \Delta) \to \mathsf{Tm}\ \Gamma\ (A\,[\sigma]) \to \mathsf{Sub}\ \Gamma\ (\Delta, A) \end{aligned}$$

We also want identities and composition:

$$\begin{aligned} \mathsf{id} &: \mathsf{Sub}\ \Gamma\ \Gamma \\ _;_ &: \mathsf{Sub}\ \Gamma\ \Delta \to \mathsf{Sub}\ \Delta\ \Theta \to \mathsf{Sub}\ \Gamma\ \Theta \end{aligned}$$

As well as weakening, variables, and term substitution:

$$\pi_1 : \mathsf{Sub}\ \Gamma\ (\Delta, A) \to \mathsf{Sub}\ \Gamma\ \Delta$$

$$\begin{aligned} \pi_2 &: (\sigma : \mathsf{Sub}\ \Gamma\ (\Delta, A)) \to \mathsf{Tm}\ \Gamma\ (A\,[\pi_1\,\sigma]) \\ _\,[_] &: \mathsf{Tm}\ \Delta\ A \to (\sigma : \mathsf{Sub}\ \Gamma\ \Delta) \to \mathsf{Tm}\ \Gamma\ (A\,[\sigma]). \end{aligned}$$

We also need laws, which we will introduce as equality constructors.

Equality constructors

A quotient inductive-inductive type allows us to also introduce constructors for equalities in the types we are defining:

$$
\begin{aligned}
[\mathsf{id}]_{\mathsf{Ty}} &: A\,[\mathsf{id}] =^{\mathsf{Ty}\,\Gamma} A \\
[;]_{\mathsf{Ty}} &: A\,[\sigma;\tau] =^{\mathsf{Ty}\,\Gamma} A\,[\tau]\,[\sigma] \\
[]\mathcal{U} &: \mathcal{U}\,[\sigma] =^{\mathsf{Ty}\,\Gamma} \mathcal{U} \\
\cdot\eta &: \{\sigma : \mathsf{Sub}\,\Gamma\,\cdot\} \to \sigma =^{\mathsf{Sub}\,\Gamma\,\cdot} \cdot \\
\pi\eta &: \{\sigma : \mathsf{Sub}\,\Gamma\,(\Delta, A)\} \to \sigma =^{\mathsf{Sub}\,\Gamma\,(\Delta,\,A)} (\pi_1\sigma, \pi_2\sigma) \\
\mathsf{idr} &: \sigma;\mathsf{id}_\Delta =^{\mathsf{Sub}\,\Gamma\,\Delta} \sigma \\
\mathsf{idl} &: \mathsf{id}_\Gamma;\sigma =^{\mathsf{Sub}\,\Gamma\,\Delta} \sigma \\
;\text{-}\mathsf{assoc} &: (\sigma;\tau);\gamma =^{\mathsf{Sub}\,\Gamma\,\Theta} \sigma;(\tau;\gamma)
\end{aligned}
$$

First circle of transport hell

Informally, we would expect the following equation:

$$\sigma;(\tau, t) = ((\sigma;\tau), t\,[\sigma])$$

LHS: "Replace top variable by t, then apply σ".
RHS: "Replace to variable by σ applied to t".

First circle of transport hell

Informally, we would expect the following equation:

$$\sigma;(\tau, t) = ((\sigma;\tau), t\,[\sigma])$$

LHS: "Replace top variable by t, then apply σ".
RHS: "Replace to variable by σ applied to t".

Unfortunately, this does not typecheck on the nose! We have

$$t\,[\sigma] : \mathsf{Tm}\ \Gamma\ (A\,[\tau]\,[\sigma])$$

but $(\sigma;\tau)$, _ on RHS wants something of type $\mathsf{Tm}\ \Gamma\ (A\,[\sigma;\tau])$.

First circle of transport hell

Informally, we would expect the following equation:

$$\sigma;(\tau, t) = ((\sigma;\tau), t\,[\sigma])$$

LHS: "Replace top variable by t, then apply σ".
RHS: "Replace to variable by σ applied to t".

Unfortunately, this does not typecheck on the nose! We have

$$t\,[\sigma] : \mathsf{Tm}\ \Gamma\ (A\,[\tau]\,[\sigma])$$

but $(\sigma;\tau)$, _ on RHS wants something of type $\mathsf{Tm}\ \Gamma\ (A\,[\sigma;\tau])$.

We have to use a previous equation to fix up the type:

$$\mathsf{concat} : \sigma;(\tau, t) =^{\mathsf{Sub}\,\Gamma\,(\Theta,\,A)} ((\sigma;\tau), \mathsf{transport}^{\mathsf{Tm}\,\Gamma}([;]_{\mathsf{Ty}}{}^{-1}, t\,[\sigma]))$$

Further circles

The final set of equations also need transports:

$$\pi_1\beta : \pi_1(\sigma, t) =^{\mathsf{Sub}\,\Gamma,\Delta} \sigma$$
$$\pi_2\beta : \pi_2(\sigma, t) =^{\mathsf{Tm}\,\Gamma\,(A\,[_])}_{\pi_1\beta} t$$
$$[\mathsf{id}]_{\mathsf{Tm}} : t\,[\mathsf{id}] =^{\mathsf{Tm}\,\Gamma}_{[\mathsf{id}]_{\mathsf{Ty}}} t$$
$$[;]_{\mathsf{Tm}} : t\,[\sigma;\tau] =^{\mathsf{Tm}\,\Gamma}_{[;]_{\mathsf{Ty}}} t\,[\tau]\,[\sigma]$$

This concludes the definition of the substitution calculus as a QIIT. Altenkirch and Kaposi [2016] pushes this definition further, and e.g. does Normalisation by Evaluation for it.

A gap between informal and formal reasoning

We would expect the following equation to hold

$$\pi_2\,(\sigma;\tau) = (\pi_2\,\tau)\,[\sigma]$$

and informally, we can reason as follows:

$$\begin{aligned}\pi_2\,(\sigma;\,\tau) &= \pi_2\,(\sigma;\,(\pi_1\,\tau,\,\pi_2\,\tau))\\ &= \pi_2\,(\sigma;\,\pi_1\,\tau,\,(\pi_2\,\tau)\,[\sigma])\\ &= (\pi_2\,\tau)\,[\sigma].\end{aligned}$$

A gap between informal and formal reasoning

We would expect the following equation to hold

$$\pi_2\,(\sigma;\tau) = (\pi_2\,\tau)\,[\sigma]$$

and informally, we can reason as follows:

$$\begin{aligned}\pi_2\,(\sigma;\,\tau) &= \pi_2\,(\sigma;\,(\pi_1\,\tau,\pi_2\,\tau))\\ &= \pi_2\,(\sigma;\,\pi_1\,\tau,\,(\pi_2\,\tau)\,[\sigma])\\ &= (\pi_2\,\tau)\,[\sigma].\end{aligned}$$

Formally, the equation is not even well formed. We instead can say

$$\pi_2\,(\sigma;\,\tau) =_p^{\mathsf{Tm}\,\Gamma} (\pi_2\,\tau)\,[\sigma]$$

for some $p : A\,[\pi_1(\sigma;\,\tau)] = A\,[\pi_1\tau]\,[\sigma]$, and the proof is 50% longer due to manipulating the transports occurring in types and terms.

A gap between informal and formal reasoning

We would expect the following equation to hold

$$\pi_2\,(\sigma;\tau) = (\pi_2\,\tau)\,[\sigma]$$

and informally, we can reason as follows:

$$\begin{aligned}\pi_2\,(\sigma;\,\tau) &= \pi_2\,(\sigma;\,(\pi_1\,\tau,\pi_2\,\tau))\\ &= \pi_2\,(\sigma;\,\pi_1\,\tau,\,(\pi_2\,\tau)\,[\sigma])\\ &= (\pi_2\,\tau)\,[\sigma].\end{aligned}$$

Formally, the equation is not even well formed. We instead can say

$$\pi_2\,(\sigma;\,\tau) =_p^{\mathsf{Tm}\,\Gamma} (\pi_2\,\tau)\,[\sigma]$$

for some $p : A\,[\pi_1(\sigma;\,\tau)] = A\,[\pi_1\tau]\,[\sigma]$, and the proof is 50% longer due to manipulating the transports occurring in types and terms.

Can we do better?

The Substitution Calculus as a QIIRT

Analysis: this instance of transport hell arises because of type substitutions not being definitionally equal.

What if we could arrange for them to be?

The Substitution Calculus as a QIIRT

Analysis: this instance of transport hell arises because of type substitutions not being definitionally equal.

What if we could arrange for them to be?

Following Danielsson [2007] (and Dybjer [1996]), we replace the constructor _ [_] with a simultaneously recursively defined function

$$_\,[_] : \mathsf{Ty}\ \Delta \to \mathsf{Sub}\ \Gamma\ \Delta \to \mathsf{Ty}\ \Gamma$$

The resulting definition is no longer a Quotient Inductive-Inductive Type, but a Quotient Inductive-Inductive-Recursive Type.

Agda can be convinced to accept our definition (using `REWRITE` rules), but such definitions are not foundationally well understood — a challenge!

The need for overlapping patterns

First attempt:

$$
\begin{aligned}
_\,[_]_{\mathsf{Ty}} &: \mathsf{Ty}\ \Delta \to \mathsf{Sub}\ \Gamma\ \Delta \to \mathsf{Ty}\ \Gamma \\
A\,[\mathsf{id}]_{\mathsf{Ty}} &\coloneqq A \\
A\,[\sigma;\tau]_{\mathsf{Ty}} &\coloneqq (A\,[\tau]_{\mathsf{Ty}})\,[\sigma]_{\mathsf{Ty}} \\
\mathcal{U}\,[\sigma]_{\mathsf{Ty}} &\coloneqq \mathcal{U}
\end{aligned}
$$

The need for overlapping patterns

First attempt:

$$
\begin{aligned}
_\,[_]_{\mathsf{Ty}} &: \mathsf{Ty}\ \Delta \to \mathsf{Sub}\ \Gamma\ \Delta \to \mathsf{Ty}\ \Gamma \\
A\,[\mathsf{id}]_{\mathsf{Ty}} &\coloneqq A \\
A\,[\sigma;\tau]_{\mathsf{Ty}} &\coloneqq (A\,[\tau]_{\mathsf{Ty}})\,[\sigma]_{\mathsf{Ty}} \\
\mathcal{U}\,[\sigma]_{\mathsf{Ty}} &\coloneqq \mathcal{U}
\end{aligned}
$$

Unfortunately, due to the translation to case trees/eliminators, the equation

$$\mathcal{U}\,[\sigma]_{\mathsf{Ty}} = \mathcal{U}$$

will *not* hold definitionally.

To overcome this, we turn to a pattern matching semantics using overlapping patterns $fx \Rightarrow t$ (Cockx [2014]).

Type substitution with overlapping patterns

We also add other convenient derived rules:

$$
\begin{aligned}
_\,[_]_{\mathsf{Ty}} &: \mathsf{Ty}\,\Delta \to \mathsf{Sub}\,\Gamma\,\Delta \to \mathsf{Ty}\,\Gamma \\
A\,[\mathsf{id}]_{\mathsf{Ty}} &\Rightarrow A \\
A\,[\sigma;\tau]_{\mathsf{Ty}} &\Rightarrow A\,[\tau]_{\mathsf{Ty}}\,[\sigma]_{\mathsf{Ty}} \\
A\,[\pi_1(\sigma, t)]_{\mathsf{Ty}} &\Rightarrow A\,[\sigma]_{\mathsf{Ty}} \\
A\,[\pi_1\,(\sigma;\tau)]_{\mathsf{Ty}} &\Rightarrow A\,[\pi_1\tau]_{\mathsf{Ty}}\,[\sigma]_{\mathsf{Ty}} \\
\mathcal{U}\,[\sigma]_{\mathsf{Ty}} &\Rightarrow \mathcal{U}
\end{aligned}
$$

Type substitution with overlapping patterns

We also add other convenient derived rules:

$$
\begin{aligned}
_\,[_]_{\mathsf{Ty}} : \ & \mathsf{Ty}\,\Delta \to \mathsf{Sub}\,\Gamma\,\Delta \to \mathsf{Ty}\,\Gamma \\
A\,[\mathsf{id}]_{\mathsf{Ty}} &\Rightarrow A \\
A\,[\sigma;\tau]_{\mathsf{Ty}} &\Rightarrow A\,[\tau]_{\mathsf{Ty}}\,[\sigma]_{\mathsf{Ty}} \\
A\,[\pi_1(\sigma, t)]_{\mathsf{Ty}} &\Rightarrow A\,[\sigma]_{\mathsf{Ty}} \\
A\,[\pi_1\,(\sigma;\tau)]_{\mathsf{Ty}} &\Rightarrow A\,[\pi_1\tau]_{\mathsf{Ty}}\,[\sigma]_{\mathsf{Ty}} \\
\mathcal{U}\,[\sigma]_{\mathsf{Ty}} &\Rightarrow \mathcal{U}
\end{aligned}
$$

Proposition Type substitution $_\,[_]_{\mathsf{Ty}}$ is terminating and locally confluent.

Type substitution with overlapping patterns

We also add other convenient derived rules:

$$
\begin{aligned}
_\,[_]_{\mathsf{Ty}} &: \mathsf{Ty}\,\Delta \to \mathsf{Sub}\,\Gamma\,\Delta \to \mathsf{Ty}\,\Gamma \\
A\,[\mathsf{id}]_{\mathsf{Ty}} &\Rightarrow A \\
A\,[\sigma;\tau]_{\mathsf{Ty}} &\Rightarrow A\,[\tau]_{\mathsf{Ty}}\,[\sigma]_{\mathsf{Ty}} \\
A\,[\pi_1(\sigma, t)]_{\mathsf{Ty}} &\Rightarrow A\,[\sigma]_{\mathsf{Ty}} \\
A\,[\pi_1\,(\sigma;\tau)]_{\mathsf{Ty}} &\Rightarrow A\,[\pi_1\tau]_{\mathsf{Ty}}\,[\sigma]_{\mathsf{Ty}} \\
\mathcal{U}\,[\sigma]_{\mathsf{Ty}} &\Rightarrow \mathcal{U}
\end{aligned}
$$

Proposition Type substitution $_\,[_]_{\mathsf{Ty}}$ is terminating and locally confluent.

Proposition For every equality constructor $\mathsf{c} : \sigma = \tau$, $A\,[\sigma]_{\mathsf{Ty}} = A\,[\tau]_{\mathsf{Ty}}$.

Updated constructors

With $A\,[\sigma]_{\mathsf{Ty}}$ computing, we can now give the following simpler types to some of the constructors:

$$\begin{aligned}
\mathsf{concat} : \sigma;(\tau, t) &=^{\mathsf{Sub}\,\Gamma\,(\Theta,\,A)}\ (\sigma;\tau), t\,[\sigma] \\
\pi_2\beta : \pi_2(\sigma, t) &=^{\mathsf{Tm}\,\Gamma\,(A\,[\sigma]_{\mathsf{Ty}})}\ t \\
[\mathsf{id}]_{\mathsf{Tm}} : t\,[\mathsf{id}] &=^{\mathsf{Tm}\,\Gamma\,A}\ t \\
[;]_{\mathsf{Tm}} : t\,[\sigma;\,\tau] &=^{\mathsf{Tm}\,\Gamma\,(A\,[\sigma;\,\tau]_{\mathsf{Ty}})}\ t\,[\tau]\,[\sigma]
\end{aligned}$$

Note the absence of transport.

Adding a universe

We turn $\mathcal{U}$ into a proper universe by adding

$$\mathsf{El} : \mathsf{Tm}\,\Gamma\,\mathcal{U} \to \mathsf{Ty}\,\Gamma$$

We also need to extend the definition of type substitution:

$$(\mathsf{El}\,u)\,[\sigma]_{\mathsf{Ty}} \Rightarrow \mathsf{El}\,(u\,[\sigma])$$

Adding a universe

We turn $\mathcal{U}$ into a proper universe by adding

$$\mathsf{El} : \mathsf{Tm}\,\Gamma\,\mathcal{U} \to \mathsf{Ty}\,\Gamma$$

We also need to extend the definition of type substitution:

$$(\mathsf{El}\,u)\,[\sigma]_{\mathsf{Ty}} \Rightarrow \mathsf{El}\,(u\,[\sigma])$$

However, this breaks confluence!

$$(\mathsf{El}\,u)\,[\mathsf{id}]_{\mathsf{Ty}}$$

Adding a universe

We turn $\mathcal{U}$ into a proper universe by adding

$$\mathsf{El} : \mathsf{Tm}\,\Gamma\,\mathcal{U} \to \mathsf{Ty}\,\Gamma$$

We also need to extend the definition of type substitution:

$$(\mathsf{El}\,u)\,[\sigma]_{\mathsf{Ty}} \Rightarrow \mathsf{El}\,(u\,[\sigma])$$

However, this breaks confluence!

$$(\mathsf{El}\,u)\,[\mathsf{id}]_{\mathsf{Ty}} \Rightarrow \mathsf{El}\,(u\,[\mathsf{id}])$$

Adding a universe

We turn $\mathcal{U}$ into a proper universe by adding

$$\mathsf{El} : \mathsf{Tm}\,\Gamma\,\mathcal{U} \to \mathsf{Ty}\,\Gamma$$

We also need to extend the definition of type substitution:

$$(\mathsf{El}\,u)\,[\sigma]_{\mathsf{Ty}} \Rightarrow \mathsf{El}\,(u\,[\sigma])$$

However, this breaks confluence!

$$\mathsf{El}\,u \Leftarrow (\mathsf{El}\,u)\,[\mathsf{id}]_{\mathsf{Ty}} \Rightarrow \mathsf{El}\,(u\,[\mathsf{id}])$$

and $u \not\equiv u\,[\mathsf{id}]$.

Adding a universe

We turn $\mathcal{U}$ into a proper universe by adding

$$\mathsf{El} : \mathsf{Tm}\,\Gamma\,\mathcal{U} \to \mathsf{Ty}\,\Gamma$$

We also need to extend the definition of type substitution:

$$(\mathsf{El}\,u)\,[\sigma]_{\mathsf{Ty}} \Rightarrow \mathsf{El}\,(u\,[\sigma])$$

However, this breaks confluence!

$$\mathsf{El}\,u \Leftarrow (\mathsf{El}\,u)\,[\mathsf{id}]_{\mathsf{Ty}} \Rightarrow \mathsf{El}\,(u\,[\mathsf{id}])$$

and $u \not\equiv u\,[\mathsf{id}]$.

Solution: We make a smart constructor for $u\,[\sigma]$, which computes.

Smart term substitution

$$
\begin{aligned}
_\,[_]_{\mathsf{Tm}} &: \mathsf{Tm}\,\Delta\,A \to (\sigma : \mathsf{Sub}\,\Gamma\,\Delta) \to \mathsf{Tm}\,\Gamma\,(A\,[\sigma]_{\mathsf{Ty}}) \\
t\,[\mathsf{id}]_{\mathsf{Tm}} &\Rightarrow t \\
t\,[\sigma;\tau]_{\mathsf{Tm}} &\Rightarrow t\,[\tau]_{\mathsf{Tm}}\,[\sigma]_{\mathsf{Tm}} \\
t\,[\pi_1(\sigma,s)]_{\mathsf{Tm}} &\Rightarrow t\,[\sigma]_{\mathsf{Tm}} \\
t\,[\pi_1(\sigma;\tau)]_{\mathsf{Tm}} &\Rightarrow t\,[\pi_1\tau]_{\mathsf{Tm}}\,[\sigma]_{\mathsf{Tm}} \\
t\,[\sigma]_{\mathsf{Tm}} &\Rightarrow t\,[\sigma], \quad \text{otherwise}
\end{aligned}
$$

Note: no overlapping patterns.

Smart term substitution

$$
\begin{aligned}
_\,[_]_{\mathsf{Tm}} &: \mathsf{Tm}\,\Delta\,A \to (\sigma : \mathsf{Sub}\,\Gamma\,\Delta) \to \mathsf{Tm}\,\Gamma\,(A\,[\sigma]_{\mathsf{Ty}}) \\
t\,[\mathsf{id}]_{\mathsf{Tm}} &\Rightarrow t \\
t\,[\sigma;\tau]_{\mathsf{Tm}} &\Rightarrow t\,[\tau]_{\mathsf{Tm}}\,[\sigma]_{\mathsf{Tm}} \\
t\,[\pi_1(\sigma,s)]_{\mathsf{Tm}} &\Rightarrow t\,[\sigma]_{\mathsf{Tm}} \\
t\,[\pi_1(\sigma;\tau)]_{\mathsf{Tm}} &\Rightarrow t\,[\pi_1\tau]_{\mathsf{Tm}}\,[\sigma]_{\mathsf{Tm}} \\
t\,[\sigma]_{\mathsf{Tm}} &\Rightarrow t\,[\sigma], \quad \text{otherwise}
\end{aligned}
$$

Note: no overlapping patterns.

We define $(\mathsf{El}\,u)\,[\sigma]_{\mathsf{Ty}} \Rightarrow \mathsf{El}\,(u\,[\sigma]_{\mathsf{Tm}})$. Of course, we have to (re)check termination, confluence and respecting equality.

Smart term substitution

$$
\begin{aligned}
_\,[_]_{\mathsf{Tm}} &: \mathsf{Tm}\,\Delta\,A \to (\sigma : \mathsf{Sub}\,\Gamma\,\Delta) \to \mathsf{Tm}\,\Gamma\,(A\,[\sigma]_{\mathsf{Ty}}) \\
t\,[\mathsf{id}]_{\mathsf{Tm}} &\Rightarrow t \\
t\,[\sigma;\tau]_{\mathsf{Tm}} &\Rightarrow t\,[\tau]_{\mathsf{Tm}}\,[\sigma]_{\mathsf{Tm}} \\
t\,[\pi_1(\sigma,s)]_{\mathsf{Tm}} &\Rightarrow t\,[\sigma]_{\mathsf{Tm}} \\
t\,[\pi_1(\sigma;\tau)]_{\mathsf{Tm}} &\Rightarrow t\,[\pi_1\tau]_{\mathsf{Tm}}\,[\sigma]_{\mathsf{Tm}} \\
t\,[\sigma]_{\mathsf{Tm}} &\Rightarrow t\,[\sigma], \quad \text{otherwise}
\end{aligned}
$$

Note: no overlapping patterns.

We define $(\mathsf{El}\,u)\,[\sigma]_{\mathsf{Ty}} \Rightarrow \mathsf{El}\,(u\,[\sigma]_{\mathsf{Tm}})$. Of course, we have to (re)check termination, confluence and respecting equality.

Proposition For all σ and t, $t\,[\sigma]_{\mathsf{Tm}} = t\,[\sigma]$ (but not $t\,[\sigma]_{\mathsf{Tm}} \equiv t\,[\sigma]$ in general).

Adding Pi types

Next we add function types:

$$
\begin{aligned}
\Pi &: (A : \mathsf{Ty}\,\Gamma) \to \mathsf{Ty}\,(\Gamma, A) \to \mathsf{Ty}\,\Gamma \\
\lambda &: \mathsf{Tm}\,(\Gamma, A)\,B \to \mathsf{Tm}\,\Gamma\,(\Pi\,A\,B) \\
\mathsf{app} &: \mathsf{Tm}\,\Gamma\,(\Pi\,A\,B) \to \mathsf{Tm}\,(\Gamma, A)\,B \\
\Pi\beta &: \mathsf{app}\,(\lambda\,t) =^{\mathsf{Tm}\,\Gamma\,A} t \\
\Pi\eta &: t =^{\mathsf{Tm}\,\Gamma\,A} \lambda\,(\mathsf{app}\,t)
\end{aligned}
$$

Adding Pi types

Next we add function types:

$$
\begin{aligned}
\Pi &: (A : \mathsf{Ty}\,\Gamma) \to \mathsf{Ty}\,(\Gamma, A) \to \mathsf{Ty}\,\Gamma \\
\lambda &: \mathsf{Tm}\,(\Gamma, A)\,B \to \mathsf{Tm}\,\Gamma\,(\Pi\,A\,B) \\
\mathsf{app} &: \mathsf{Tm}\,\Gamma\,(\Pi\,A\,B) \to \mathsf{Tm}\,(\Gamma, A)\,B \\
\Pi\beta &: \mathsf{app}\,(\lambda\,t) =^{\mathsf{Tm}\,\Gamma\,A} t \\
\Pi\eta &: t =^{\mathsf{Tm}\,\Gamma\,A} \lambda\,(\mathsf{app}\,t)
\end{aligned}
$$

Confluence again breaks if we use the naive extension to type substitution

$$(\Pi\,A\,B)\,[\sigma]_{\mathsf{Ty}} =^{\mathsf{Ty}\,\Gamma} \Pi\,(A\,[\sigma]_{\mathsf{Ty}})\,(B\,[\sigma \uparrow A]_{\mathsf{Ty}})$$

where $\sigma \uparrow A = ((\pi_1\mathsf{id};\,\sigma), \pi_2\mathsf{id}) : \mathsf{Sub}\,(\Gamma, A\,[\sigma]_{\mathsf{Ty}})\,(\Delta, A)$.

Substitution lifting

Instead we also compute the lifting recursively:

$$
\begin{aligned}
\uparrow &: (\sigma : \mathsf{Sub}\,\Gamma\,\Delta)(A : \mathsf{Ty}\,\Delta) \to \mathsf{Sub}\,(\Gamma, A\,[\sigma]_{\mathsf{Ty}})\,(\Delta, A) \\
\mathsf{id}\uparrow A &\Rightarrow \mathsf{id} \\
(\sigma\ ;\ \tau)\uparrow A &\Rightarrow (\sigma\uparrow\tau\,[A]_{\mathsf{Ty}})\ ;\ (\tau\uparrow A) \\
\pi_1(\sigma, t)\uparrow A &\Rightarrow \sigma\uparrow A \\
\pi_1(\sigma;\tau)\uparrow A &\Rightarrow (\sigma\uparrow\pi_1\tau\,[A]_{\mathsf{Ty}});(\pi_1\tau\uparrow A) \\
\sigma\uparrow A &\Rightarrow \sigma^+, \quad \text{otherwise}
\end{aligned}
$$

where $\sigma^+ :\equiv ((\pi_1\mathsf{id};\sigma), \pi_2\mathsf{id})$.

Substitution lifting

Instead we also compute the lifting recursively:

$$
\begin{aligned}
\uparrow &: (\sigma : \mathsf{Sub}\,\Gamma\,\Delta)(A : \mathsf{Ty}\,\Delta) \to \mathsf{Sub}\,(\Gamma, A\,[\sigma]_{\mathsf{Ty}})\,(\Delta, A) \\
\mathsf{id}\uparrow A &\Rightarrow \mathsf{id} \\
(\sigma\ ;\ \tau)\uparrow A &\Rightarrow (\sigma\uparrow\tau\,[A]_{\mathsf{Ty}})\ ;\ (\tau\uparrow A) \\
\pi_1(\sigma, t)\uparrow A &\Rightarrow \sigma\uparrow A \\
\pi_1(\sigma;\tau)\uparrow A &\Rightarrow (\sigma\uparrow\pi_1\tau\,[A]_{\mathsf{Ty}}); (\pi_1\tau\uparrow A) \\
\sigma\uparrow A &\Rightarrow \sigma^+, \quad \text{otherwise}
\end{aligned}
$$

where $\sigma^+ :\equiv ((\pi_1\mathsf{id};\sigma), \pi_2\mathsf{id})$.
We define

$$(\Pi\,A\,B)\,[\sigma]_{\mathsf{Ty}} \Rightarrow \Pi\,(A\,[\sigma]_{\mathsf{Ty}})\,(B\,[\sigma\uparrow A]_{\mathsf{Ty}})$$

and add an equality constructor for term substitution (no transport needed!):

$$[]\lambda : (\lambda\,t)\,[\sigma] =^{\mathsf{Tm}\,\Gamma\,((\Pi\,A\,B)\,[\sigma]_{\mathsf{Ty}})} \lambda\,(t\,[\sigma\uparrow A])$$

Properties

Lemma We have $\mathsf{id}^+ = \mathsf{id}$ and $(\sigma \mathbin{;} \tau)^+ = \sigma^+ \mathbin{;} \tau^+$.

Proposition We have $\sigma{\uparrow}A = \sigma^+$ for all σ and A.

Properties

Lemma We have $\mathsf{id}^+ = \mathsf{id}$ and $(\sigma \mathbin{;} \tau)^+ = \sigma^+ \mathbin{;} \tau^+$.

Proposition We have $\sigma{\uparrow}A = \sigma^+$ for all σ and A.

Proposition Type substitution $_\,[_]_{\mathsf{Ty}}$ is terminating and locally confluent.

Properties

Lemma We have $\mathsf{id}^+ = \mathsf{id}$ and $(\sigma \mathbin{;} \tau)^+ = \sigma^+ \mathbin{;} \tau^+$.

Proposition We have $\sigma{\uparrow}A = \sigma^+$ for all σ and A.

Proposition Type substitution $_\,[_]_{\mathsf{Ty}}$ is terminating and locally confluent.

To prove that $_\,[_]_{\mathsf{Ty}}$ respect equalities, we need to further generalise $\sigma{\uparrow}A$ to lifting an entire telescope $\sigma{\uparrow\uparrow}A_1, \ldots, A_n$.

Properties

Lemma We have $\mathsf{id}^+ = \mathsf{id}$ and $(\sigma \mathbin{;} \tau)^+ = \sigma^+ \mathbin{;} \tau^+$.

Proposition We have $\sigma{\uparrow}A = \sigma^+$ for all σ and A.

Proposition Type substitution $_\,[_]_{\mathsf{Ty}}$ is terminating and locally confluent.

To prove that $_\,[_]_{\mathsf{Ty}}$ respect equalities, we need to further generalise $\sigma{\uparrow}A$ to lifting an entire telescope $\sigma{\uparrow\uparrow}A_1, \ldots, A_n$.

Proposition For every equality constructor $\mathsf{c} : \sigma = \tau$, we have

$$\Xi\,[\sigma]_{\mathsf{Tel}} = \Xi\,[\tau]_{\mathsf{Tel}} \qquad \sigma \mathbin{\uparrow\uparrow} \Xi =^{\mathsf{Sub}\ (\Gamma +\!\!+ \Xi\,[_]_{\mathsf{Tel}})\ (\Delta +\!\!+ \Xi)}_{\mathsf{c}} \tau \mathbin{\uparrow\uparrow} \Xi$$

$$A\,[\sigma \mathbin{\uparrow\uparrow} \Xi]_{\mathsf{Ty}} =^{\mathsf{Ty}\ (\Gamma +\!\!+ \Xi\,[_]_{\mathsf{Tel}})}_{\mathsf{c}} A\,[\tau \mathbin{\uparrow\uparrow} \Xi]_{\mathsf{Ty}} \qquad A\,[\sigma]_{\mathsf{Ty}} =^{\mathsf{Ty}\ \Gamma} A\,[\tau]_{\mathsf{Ty}}$$

Adding other type formers

We can also add e.g.

- extensional identity types
- a hierarchy of Coquand universes
- a unit type with an eta law

without additional difficulty.

Adding inductive types

We have not been able to add inductive types completely without transport. For example:

$$
\begin{aligned}
\mathbf{2} &: \mathsf{Ty}\ \Gamma \\
0_{\mathbf{2}} &: \mathsf{Tm}\ \Gamma\ \mathbf{2} \\
1_{\mathbf{2}} &: \mathsf{Tm}\ \Gamma\ \mathbf{2} \\
\mathsf{elim}_{\mathbf{2}} &: (P : \mathsf{Ty}\ (\Gamma, \mathbf{2})) \to \mathsf{Tm}\ \Gamma\ (P\,[\langle\, 0_{\mathbf{2}} \,\rangle]_{\mathsf{Ty}}) \to \mathsf{Tm}\ \Gamma\ (P\,[\langle\, 1_{\mathbf{2}} \,\rangle]_{\mathsf{Ty}}) \\
&\quad \to (b : \mathsf{Tm}\ \Gamma\ \mathbf{2}) \to \mathsf{Tm}\ \Gamma\ (P\,[\langle\, b \,\rangle]_{\mathsf{Ty}})
\end{aligned}
$$

where $\langle\, t \,\rangle :\equiv (\mathsf{id}, t) : \mathsf{Sub}\ \Gamma\ (\Gamma, A)$.

So far so good. Type substitution is also okay:

$$\mathbf{2}\,[\sigma]_{\mathsf{Ty}} \Rightarrow \mathbf{2}$$

Adding inductive types: equalities

But we seem to need transport for the term substitution:

$$
\begin{aligned}
\mathbf{2}\beta_i &: \mathsf{elim}_{\mathbf{2}}\, P\, m_0\, m_1\, i = m_i \\
[]\mathsf{elim}_{\mathbf{2}} &: (\mathsf{elim}_{\mathbf{2}}\, P\, f\, t\, b)\,[\sigma] =^{\mathsf{Tm}\,\Gamma}_{p\,b} \\
&\qquad \mathsf{elim}_{\mathbf{2}}\,(P\,[\sigma^+])\;(\mathsf{transport}^{\mathsf{Tm}\,\Gamma}(p\,0_{\mathbf{2}}, f\,[\sigma]))\\
&\qquad\qquad (\mathsf{transport}^{\mathsf{Tm}\,\Gamma}(p\,1_{\mathbf{2}}, t\,[\sigma]))\;(b\,[\sigma])
\end{aligned}
$$

where $p\,x : P\,[\langle\, x\,[\sigma]\,\rangle; \sigma^+]_{\mathsf{Ty}} = P\,[\sigma; \langle\, x\,\rangle]_{\mathsf{Ty}}$ — there does not seem to be an obvious way to direct this equation into a clause of $_\,[_]_{\mathsf{Ty}}$.

Hence, we can get away with fewer transports in the syntax, but we have not been able to banish them all (note: in QIIT version, also $\mathbf{2}\beta_i$ requires transport).

Elimination principle

Given *motive* type families

$$
\begin{aligned}
\mathsf{Ctx}^M &: \mathsf{Ctx} \to \mathsf{Set} \\
\mathsf{Sub}^M &: \mathsf{Ctx}^M\ \Gamma \to \mathsf{Ctx}^M\ \Delta \to \mathsf{Sub}\ \Gamma\ \Delta \to \mathsf{Set} \\
\mathsf{Ty}^M &: \mathsf{Ctx}^M\ \Gamma \to \mathsf{Ty}\ \Gamma \to \mathsf{Set} \\
\mathsf{Tm}^M &: (\Gamma^M : \mathsf{Ctx}^M\ \Gamma) \to \mathsf{Ty}^M\ \Gamma^M\ A \to \mathsf{Tm}\ \Gamma\ A \to \mathsf{Set}
\end{aligned}
$$

and *methods* for each constructor, the elimination principle gives us

$$
\begin{aligned}
\mathsf{elim}_{\mathsf{Ctx}} &: (\Gamma : \mathsf{Ctx}) \to \mathsf{Ctx}^M\ \Gamma \\
\mathsf{elim}_{\mathsf{Sub}} &: (\sigma : \mathsf{Sub}\ \Gamma\ \Delta) \to \mathsf{Sub}^M\ (\mathsf{elim}_{\mathsf{Ctx}}\ \Gamma)\ (\mathsf{elim}_{\mathsf{Ctx}}\ \Delta)\ \sigma \\
\mathsf{elim}_{\mathsf{Ty}} &: (A : \mathsf{Ty}\ \Gamma) \to \mathsf{Ty}^M\ (\mathsf{elim}_{\mathsf{Ctx}}\ \Gamma)\ A \\
\mathsf{elim}_{\mathsf{Tm}} &: (t : \mathsf{Tm}\ \Gamma\ A) \to \mathsf{Tm}^M\ (\mathsf{elim}_{\mathsf{Ctx}}\ \Gamma)\ (\mathsf{elim}_{\mathsf{Ty}}\ A)\ t.
\end{aligned}
$$

Standard model

We can choose

$$
\begin{aligned}
\mathsf{Ctx}^M\ _ &:\equiv \mathsf{Set} \\
\mathsf{Sub}^M\ \Gamma^M\ \Delta^M\ _ &:\equiv \Gamma^M \to \Delta^M \\
\mathsf{Ty}^M\ \Gamma^M\ _ &:\equiv \Gamma^M \to \mathsf{Set} \\
\mathsf{Tm}^M\ \Gamma^M\ A^M\ _ &:\equiv (\gamma : \Gamma^M) \to A^M\ \gamma \\
&\vdots
\end{aligned}
$$

This interprets object types as sets, object function types as function types, etc.

Comparing with the QIIT version

We can also choose

$$\begin{aligned}
\mathsf{Ctx}^M\ _ &:\equiv \mathsf{Ctx}_{\mathsf{QII}} \\
\mathsf{Sub}^M\ \Gamma^M\ \Delta^M\ _ &:\equiv \mathsf{Sub}_{\mathsf{QII}}\ \Gamma^M\ \Delta^M \\
\mathsf{Ty}^M\ \Gamma^M\ _ &:\equiv \mathsf{Ty}_{\mathsf{QII}}\ \Gamma^M \\
\mathsf{Tm}^M\ \Gamma^M\ A^M\ _ &:\equiv \mathsf{Tm}_{\mathsf{QII}}\ \Gamma^M\ A^M
\end{aligned}$$

and vice versa to translate between the QIIRT and QIIT version.

Theorem Type theory defined as a QIIRT and a QIIT are equivalent, i.e.

- $\mathsf{Ctx}_{\mathsf{QIIR}} \simeq \mathsf{Ctx}_{\mathsf{QII}}$,
- $\big((\Gamma : \mathsf{Ctx}_{\mathsf{QIIR}}) \to \mathsf{Ty}_{\mathsf{QIIR}}\ \Gamma\big) \simeq \big((\Gamma : \mathsf{Ctx}_{\mathsf{QII}}) \to \mathsf{Ty}_{\mathsf{QII}}\ \Gamma\big)$,
- $((\Gamma\ \Delta : \mathsf{Ctx}_{\mathsf{QIIR}}) \to \mathsf{Sub}_{\mathsf{QIIR}}\ \Gamma\ \Delta) \simeq ((\Gamma\ \Delta : \mathsf{Ctx}_{\mathsf{QII}}) \to \mathsf{Sub}_{\mathsf{QII}}\ \Gamma\ \Delta)$, and
- $\big((\Gamma : \mathsf{Ctx}_{\mathsf{QIIR}}) \to (A : \mathsf{Ty}_{\mathsf{QIIR}}\ \Gamma) \to \mathsf{Tm}_{\mathsf{QIIR}}\ \Gamma\ A\big) \simeq$
  $\big((\Gamma : \mathsf{Ctx}_{\mathsf{QII}}) \to (A : \mathsf{Ty}_{\mathsf{QII}}\ \Gamma) \to \mathsf{Tm}_{\mathsf{QII}}\ \Gamma\ A\big)$.

Summary and conclusions

Quotient inductive-inductive types can be used to conveniently represent type theory in type theory.

Quotient inductive-inductive-recursive types can be used to avoid much of the transport hell that arises from QIITs alone.

NbE using QIIRTs around 20% shorter than corresponding QIIT version.

Current justification for QIIRTs unclear; more research needed.

Summary and conclusions

Quotient inductive-inductive types can be used to conveniently represent type theory in type theory.

Quotient inductive-inductive-recursive types can be used to avoid much of the transport hell that arises from QIITs alone.

NbE using QIIRTs around 20% shorter than corresponding QIIT version.

Current justification for QIIRTs unclear; more research needed.

Thanks!

References

- Thorsten Altenkirch, Paolo Capriotti, Gabe Dijkstra, Nicolai Kraus, and Fredrik Nordvall Forsberg. 2018. "Quotient inductive-inductive types". In *Foundations of Software Science and Computation Structures (FoSSaCS '18)*, 293–310. DOI: 10.1007/978-3-319-89366-2_16.
- Thorsten Altenkirch and Ambrus Kaposi. 2016. "Type theory in type theory using quotient inductive types". In *Principles of Programming Languages (POPL '16)*, 18-–29. DOI: 10.1145/2837614.2837638.
- Thorsten Altenkirch and Ambrus Kaposi. 2016. "Normalisation by Evaluation for Dependent Types". In *Formal Structures for Computation and Deduction (FSCD '16)*, 6:1–6:16. DOI: 10.4230/LIPIcs.FSCD.2016.6.
- James Chapman. 2009. "Type Theory Should Eat Itself". In *Workshop on Mathematically Structured Functional Programming (MSFP '06)*, 21–36. DOI: 10.1016/j.entcs.2008.12.114.
- Jesper Cockx, Frank Piessens, and Dominique Devriese. 2014. "Overlapping and order-independent patterns". In *European Symposium on Programming (ESOP '14)*, 87–106. DOI: 10.1007/978-3-642-54833-8_6.
- Nils Anders Danielsson. 2007. "A formalisation of a dependently typed language as an inductive-recursive family". In *Types for Proofs and Programs (TYPES '06)*, DOI: 10.1007/978-3-540-74464-1_7.
- Peter Dybjer. 1996. "Internal type theory". In *Types for Proofs and Programs (TYPES '95)*. 120-–134. DOI: 10.1007/3-540-61780-9_66